

Coordinating with the **helios_postprocess** Project -- Summer 2026

T. Mehlhorn

2026-06-04

Contents

Coordinating with the helios_postprocess Project — Summer 2026	1
1. The shared infrastructure	1
2. Adiabats conventions — please read once before any cross-tool comparison	2
3. Your three objectives — what’s already there for you	2
4. A coordination model that I think could work	3
5. Where your work helps the calibration effort	4
6. Communication	4

Coordinating with the **helios_postprocess** Project — Summer 2026

To: Kyle
Cc: Will
From: Tom Mehlhorn
Date: June 4, 2026

Kyle, welcome to the summer project. This is a sketch of how I’m thinking about coordination between your work in Denver and the **helios_postprocess** pipeline that Will and I have been building. It’s a suggestion, not a prescription — please push back on anything that doesn’t fit how you actually want to work.

The three of us share goals but work independently, with Will as the technical lead. The pipeline is on GitHub; results land in SharePoint. You and Will both already use Claude at Xcimer. The question is how to keep our work composable across those boundaries.

1. The shared infrastructure

Everything below is already in `tamehlhorn/helios_postprocess` on GitHub. You have access. Pulling the latest gets you:

- **helios_postprocess/** — the Python pipeline. `HeliosRun` reads EXODUS, `build_run_data` produces an `ICFRunData` dataclass with ~80 attributes, `ICFAnalyzer` populates derived quantities, `ICFPlotter` writes the report PDF, `ICFOutputGenerator` writes the text summary + history CSV.
- **examples/run_analysis.py** — main entry point. Takes a base path (no extension), derives `.exo`, `.rhw`, `_published.json`, and writes `_report.pdf`, `_comparison.pdf`, `_summary.txt`, `_history.csv`.
- **examples/scan_summary.py** — scan-level CSV, one row per run, ranking against a target velocity (extendable to other targets).

- `examples/plot_shock_trajectories.py`, `examples/energy_balance_diagnostic.py`, `examples/dump_per_zone_burn_share.py` — standalone diagnostics.
- `CLAUDE.md` — long-form project guide. Auto-loaded by Claude when you start a session in the repo. ~70 pages covering EXODUS conventions, physics conventions, calibration history, known gotchas, and worked examples. This is the team’s shared memory. Any session-specific context you don’t put here is local to your own Claude project.
- `docs/Xcimer_PDD_calibration_report.{md,docx,pdf}` and the HDD companion — the two closed calibration writeups. State of the art on each target as of June 3 2026.
- `docs/Helios_PDD_HDD_summary_for_Will.pptx` — the briefing deck from June 3; top-down view of where we are.

If you want a fast tour: read `CLAUDE.md` sections “Class-Based Pipeline”, “Physics Conventions” 1-17 (especially 9 = adiabat, 5b = fusion rates), “Shock train pipeline”, “Comparison Framework”, then skim the two `Xcimer_*_calibration_report.md` files.

2. Adiabat conventions — please read once before any cross-tool comparison

This is the one thing I’d flag as non-obvious. The pipeline carries five adiabat conventions because the same `.exo` gives different numbers depending on which `n_e` model and which time-aggregation you use, and those differences are 1.5× to 14×, not 5%.

Convention	Pipeline attr	Use when comparing to
Lindl peak v	<code>data.adiabat</code>	Olson PDD references
Lindl base (at breakout)	<code>data.adiabat_at_breakout</code>	foot-shock adiabat sensitivity
RHINO partially_ionized at CR=1.5	<code>data.adiabat_min_rhino</code>	Thomas Vulcan HDD reference; HYDRA
RHINO fully_ionized_dt at CR=1.5	<code>data.adiabat_min_rhino_fully_ionized_dt</code>	Will’s native RHINO postprocess
Proper Fermi mass-avg	<code>data.adiabat_min_rhino_formula</code>	Dagnostic only

Pick one per study and document it. The convention story closed June 3 when we matched Will’s native RHINO 4.125 to 0.1% (4.13) on `WT_cthomas_base` line by switching to the `fully_ionized_dt` mode. The math is written up in `CLAUDE.md` “Adiabat convention story” and in the HDD report §5.4.

3. Your three objectives — what’s already there for you

(1) Euler scale + shock timing

The shock-train pipeline (`pressure_gradients.py` + `_compute_shock_train`) auto-detects foot / ramp / peak events, consolidates them within `min_separation_ns`, and writes per-event scalars. Set target times in `<base>_published.json`:

```
"t_foot_shock_breakout_ns": [7.5, 0.3],
"t_ramp_shock_breakout_ns": [10.0, 0.3],
"t_peak_shock_breakout_ns": [13.0, 0.3]
```

...and the comparison block prints Δ -vs-target. The R-T overlay PDF page is the visual diagnostic. `scan_summary.py` ranks candidates.

Two things to know if you use this: - Euler scaling needs geometry rescaling too ($L_{\text{new}}/L_{\text{old}} = (E_{\text{new}}/E_{\text{old}})^{(1/3)}$), not just power scaling. - EOS / opacity tables aren’t scale-invariant — adiabat history at 8 MJ won’t reproduce a 4 MJ run even with proper scaling.

(2) Pulse optimization, burn off until late

<base>_published.json accepts ~20 metric keys with [value, uncertainty] pairs. The comparison framework reports Δ for each, and scan_summary.py ranks by composite weighted distance. For burn-off design phase, fill the implosion-side keys (peak_velocity_kms, adiabat, ifar, hydro_efficiency_pct, inflight_KE_kJ, fraction_absorbed_pct, rhoR_cf, CR_max, and the three shock times above) and skip the burn-side ones.

One non-obvious gotcha worth flagging: the cost surface is not smooth. Our May 30 PDD design study found a two-plateau staircase (c33/c28 at 55 MJ / 25% foam; c25/c20 at 69 MJ / 31%; jump unlocked by spot+focus combination, not cone alone). Gradient-descent stalls on plateaus. LHS or Bayesian optimization beats it.

If you write a cost_function.py wrapper around _summary.txt and an SDK call to scipy.optimize / skopt, that would be genuinely useful infrastructure — we’d use it for HDD picket optimization too. See §5.

(3) Tritium-lean — gain + ρR_D

Two gaps in the current pipeline you’ll hit:

1. DD branches are not loaded into data_builder.py. EXODUS has TimeIntFusionProd_He4_0352 (DT alphas), TimeIntFusionProd_He4_0367 (DD alphas), and per-zone variants. They’re on the wishlist in CLAUDE.md “Boundary-tally” section. Roughly 50 lines to wire + verify with a zero-D Bosch-Hale check (the pattern is in examples/verify_zero_d.py).
2. Species-resolved ρR isn’t a scalar yet. Pattern after pressure_gradients.py; ~80 lines. Gives data.rhoR_D_history
 - data.rhoR_D_peak.

Both of these would be genuinely useful additions for the shared pipeline — see §5.

Helios fusion-rate convention update (May 27 2026): FusionRate_DT_nHe4 is reactions/s per gram (intensive), not per zone. Mass-weighting is mandatory. Same will apply to DD when wired. See CLAUDE.md Physics Conv. 5b for the worked example — mis-handling this introduces a factor of mass-of-fuel error.

4. A coordination model that I think could work

You’ll work mostly in your own local Project / repo / sim directories. That’s the right way to move fast. The question is when work flows back to the shared repo, and how we keep our project memories in sync.

Code

A possible split:

- **helios_postprocess** GitHub repo is the shared brain for *general- purpose pipeline capability*: new diagnostics, bug fixes, EXODUS-variable loaders, conventions. Both of us pull, both push to feature branches, merge to main when something is solid and reviewed. You don’t need my approval on every commit — feature branches are yours until you’re ready.
- Your local repos and sim directories stay yours: study-specific scripts, .rhw configurations, sim runs, intermediate analyses. They never need to come upstream.

The natural trigger for “land upstream” is: *Does this help anyone but me?* DD-branch loading does. A one-off plotting script for a specific study doesn’t.

If you’d rather work on your own fork and PR in chunks at convenient breakpoints, that works too. Whatever cadence feels right.

Claude / project memory

CLAUDE.md in the repo is the team's shared brain. When you discover something subtle in the pipeline — a non-obvious convention, a gotcha, a calibration sensitivity — please update CLAUDE.md along with the code change. That way the next Claude session anyone runs (yours, mine, Will's) sees the new context automatically.

Your *personal* memory (your Claude project's MEMORY.md and detail files) stays yours. Same for mine. The two don't need to merge — only the shared parts (code, CLAUDE.md, docs) need to.

Results

Anything you'd want Will or me to look at lands in SharePoint following the existing naming pattern:

```
Xcimer_PDD_calibration_report.{md,docx,pdf}  
Xcimer_HDD_calibration_report.{md,docx,pdf}  
Helios_PDD_HDD_summary_for_Will.pptx
```

If you produce, say, a tritium-lean writeup or an Euler-scaled parameter study, naming it `Xcimer_tritium_lean_study.{md,docx}` or similar would put it alongside ours in the listing.

.exo files are too large for GitHub but fine on SharePoint if you want to share a specific run for cross-checking.

5. Where your work helps the calibration effort

I want to flag this explicitly because the value flow isn't only "we hand Kyle tools." Each of your three objectives produces something we'd use on our side too:

- Euler-scaled multi-energy database (obj 1): any HDD recipe we develop at 4 MJ becomes more credible if you've shown the same pulse shape and timing logic work at 3 / 5 / 6 MJ. A scan you've already done would save us a separate run for that purpose.
- Pulse-optimization infrastructure (obj 2): a `cost_function.py`
 - Bayesian-optimization wrapper around the existing pipeline would automate something we do manually right now. If you build that and land it upstream, we'd use it for HDD picket tuning and probably for closing the PDD compression-channel residual that's still open on our side.
- DD-branch loading + species-resolved ρR (obj 3): these have been on our pipeline wishlist for a while. The DD branches in particular matter if Cliff's HYDRA-vs-Helios cross-checks ever want to include secondary reaction products. Species ρR would let us go back and audit fuel-composition assumptions on existing runs.

If any of these land in the shared repo, please flag it to me — I'll re-extract our calibration anchors with the new tooling and update the HDD / PDD reports.

In the other direction: there's an open thread from June 4 that you might find interesting depending on where objective 1 leads. RHINO's `assembled_mass` diagnostic shows our HDD production anchor (`WT_cthomas_baseline_picket_012`) at 2.38 mg vs Thomas's 3.0 mg (−21%), while the no-picket baseline matches at 3.02 mg (+0.7%). Open question is whether picket-strengthened foot shock blows mass through the `rho_peak/e` threshold, or whether it's a stagnation-time-definition artifact between RHINO's (shell-velocity min) and our default (HS-radius min). Will and I are talking about it. If you're running shock-timed pulses anyway, the assembled-mass history might show something.

6. Communication

I don't want to over-formalize this. I'd suggest:

- Use Claude in your repo checkout as your first stop — CLAUDE.md loads automatically and you'll get answers grounded in the conventions and history.

- For things Claude can't answer or that need a human call, Will is the technical lead and the natural first call. Ping me directly if it's pipeline-related and Will would rather not arbitrate.
- For broader strategy or "what does this mean for Alison," let's do occasional Zoom — happy to set one up after you've had a week to orient.

If a weekly written status of some kind (one paragraph, in notes/) makes sense from your end, I'd read those. If not, no pressure.

Looking forward to it.

— Tom